Technical Design Document

IntraCore Solutions
Voice Assistant for IT Ticketing and Identity Verification
Version 1.0.0
March 5, 2026

Nutchanon Phongoen Supachai Kheawjuy

100x

Abstract

This document describes the technical design of a voice assistant for IT ticketing and identity verification, built for IntraCore Solutions as a Cognigy Capstone Sprint project. The system combines Cognigy.AI conversational AI with xApps for visual interaction, backed by an IntraCore API that integrates with Postgres for persistent data. Users can verify their identity via voice (ANI/phone, email, employee number), manage internal support tickets, and access FAQ content through WebRTC or dial-in voice channels in multiple languages.

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Audience . . . 3

2 Business Context **3**

3 Capstone Sprint Scope **3**

4 Technology Stack **3**

5 System Overview **4**
5.1 Architecture . . . 4
5.2 Flow . . . 4
5.3 Sequence Diagrams . . . 4
5.3.1 Authentication Flow . . . 4
5.3.2 Identity and Verification Flow . . . 5
5.3.3 Ticket Creation Flow . . . 6
5.3.4 Ticket Retrieval with xApp . . . 7
5.3.5 Ticket Update Flow . . . 7
5.3.6 Close Ticket Flow . . . 8
5.3.7 List Ticket Histories with xApp . . . 8
5.3.8 Escalate Ticket Flow . . . 9

6 Data Model **10**
6.1 Entity Relationship Diagram . . . 10
6.2 companies . . . 10
6.3 employees . . . 10

6.4 tickets . . . 11
6.5 sessions . . . 11
6.6 whitelists . . . 11

7 API Design **11**
7.1 Public Endpoints . . . 11
7.2 Protected Endpoints (Bearer Auth) . . . 11
7.3 Use Case to API Mapping . . . 12
7.4 OpenAPI . . . 12

8 Identity and Verification **13**

9 Authentication **13**

10 Cognigy xApps **13**

11 Deployment **14**
11.1 Docker . . . 14
11.2 Environment . . . 14

12 CI/CD **14**

13 Security **14**

14 Deliverables Summary **14**

1 Introduction

1.1 Purpose

This Technical Design Document (TDD) provides a structured walkthrough of the Cognigy Capstone Sprint solution for IntraCore Solutions. It serves as the primary reference for understanding the architecture, APIs, data model, and deployment of the voice assistant system.

1.2 Scope

The document covers the full technical implementation: IntraCore API backend, Cognigy platform integration, xApp interfaces, identity verification, ticketing workflows, and deployment infrastructure.

1.3 Audience

This document is intended for Capstone Sprint evaluators, customer stakeholders, and technical team members who need to understand or extend the system.

2 Business Context

IntraCore Solutions is a fictional mid-sized technology firm with distributed teams across multiple locations. The organization operates a shared internal support center that handles IT requests, facilities issues, and general inquiries. The voice assistant project aims to:

- Automate common support interactions via voice
- Provide secure identity verification before sensitive operations
- Support multilingual employees (English, Thai, Japanese, German)
- Reduce load on human agents for routine tasks

3 Capstone Sprint Scope

The implemented scope includes:

- **Identity & Verification (ID&V):** Verification via ANI/phone number, email, and employee number; matches against the employees table
- **Internal Ticketing:** Open (create), Check Status, Update, Close, and Search History
- **FAQ:** Common questions answered by the voice agent
- **Voice Channels:** WebRTC and dial-in
- **Multilingual:** Support for multiple languages in both voice and xApps

4 Technology Stack

The system is built with the following technologies:

Category	Technology
Runtime	Bun
Web Framework	Elysia (v1.4)
ORM	Drizzle (v0.45)
Database	Postgres
Logging	Pino (v10)
xApp UI	Material Web (md-outlined-text-field, md-filled-button, md-outlined-select)

5 System Overview

5.1 Architecture

The system follows a three-tier architecture:

1. **Cognigy platform** — Full Cognigy.AI platform (voice agent, NLU, dialog management, Cognigy xApp); invokes IntraCore API for backend operations
2. **IntraCore API** — Elysia-based backend on Bun; validates requests, enforces auth, and performs business logic
3. **Database (Postgres)** — Persistent storage for companies, employees, tickets, sessions, and whitelists

5.2 Flow

1. User connects via WebRTC or dial-in
2. Language selection: Press 1 for English, 2 for Thai, 3 for Japanese, 4 for German
3. Cognigy authenticates via GitHub OAuth → POST /v1/auth/token
4. Cognigy calls POST /v1/auth/verify with phone, email, employee number for ID&V
5. Cognigy calls IntraCore API (create, list, update, close, history) with Bearer token
6. Cognigy xApps (verification, ticket-list, ticket-detail) are shown when needed; they use the app-page SDK to submit data back to Cognigy

5.3 Sequence Diagrams

The following diagrams depict the flow (user connects via a public phone number or WebRTC).

5.3.1 Authentication Flow

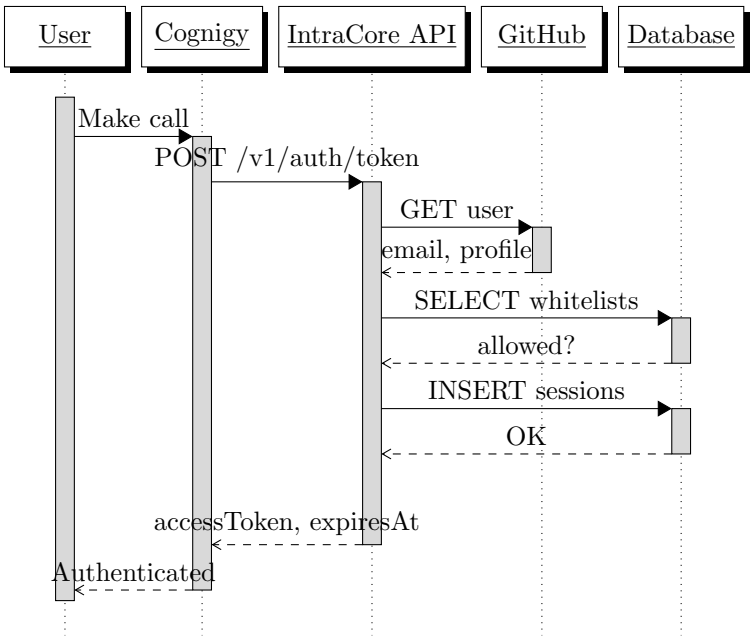

Figure 1: Authentication: User makes call to Cognigy; Cognigy calls POST /v1/auth/token; API validates with GitHub, checks whitelist, creates session; Cognigy proceeds with authenticated user. Note: githubToken is supplied by the Cognigy actor.

5.3.2 Identity and Verification Flow

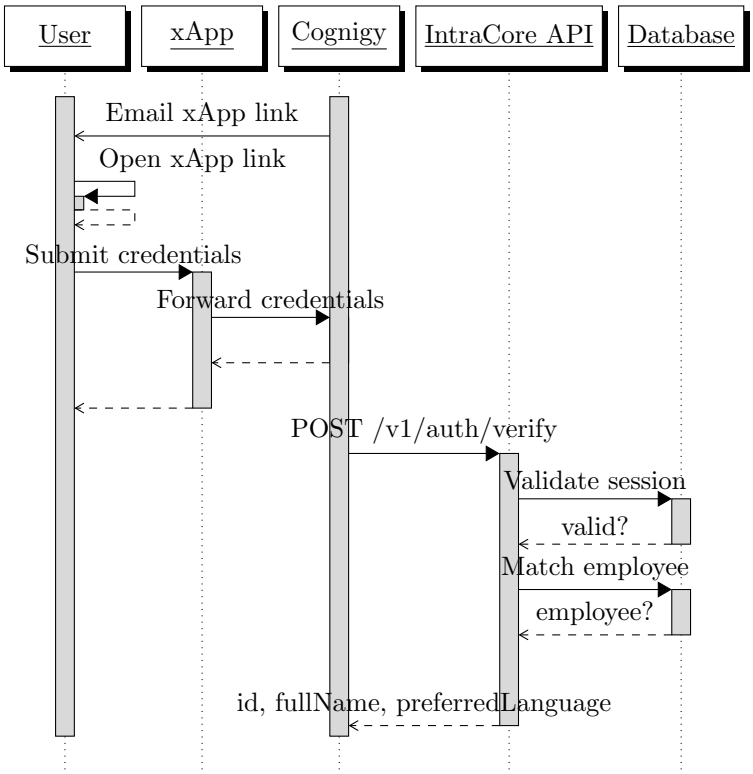

Figure 2: ID&V: User requests identity verification; Cognigy emails xApp link; user opens link, submits credentials to xApp; xApp forwards to Cognigy; API matches against employees table; Cognigy returns verified identity to user.

5.3.3 Ticket Creation Flow

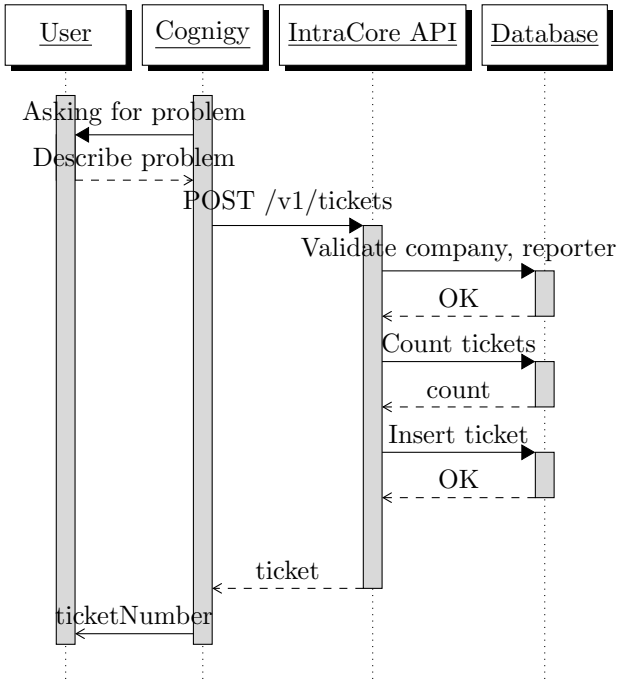

Figure 3: Ticket creation: User (authenticated) requests ticket creation; Cognigy asks for the problem; user describes the problem; Cognigy calls POST /v1/tickets; API validates and inserts ticket; Cognigy returns ticket to user.

5.3.4 Ticket Retrieval with xApp

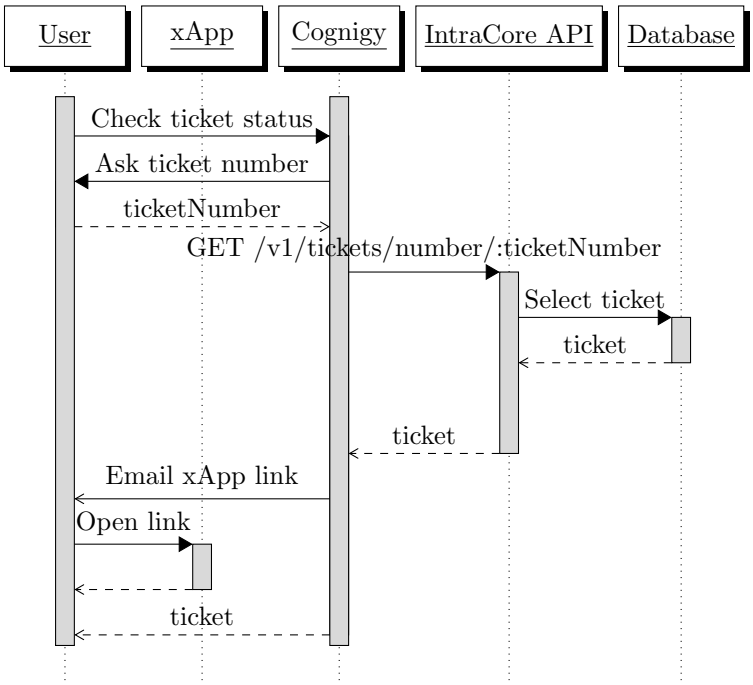

Figure 4: Ticket retrieval: User requests ticket status; Cognigy asks for ticket number; user provides it; Cognigy fetches ticket via API; Cognigy emails xApp link; user opens link to view ticket details in xApp.

5.3.5 Ticket Update Flow

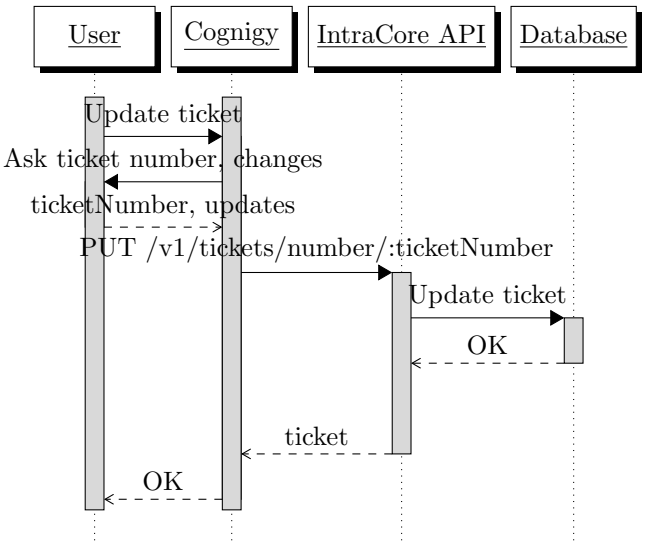

Figure 5: Ticket update: User requests ticket update; Cognigy asks for ticket number and changes; user provides; Cognigy calls PUT; API updates ticket in database; Cognigy confirms to user.

5.3.6 Close Ticket Flow

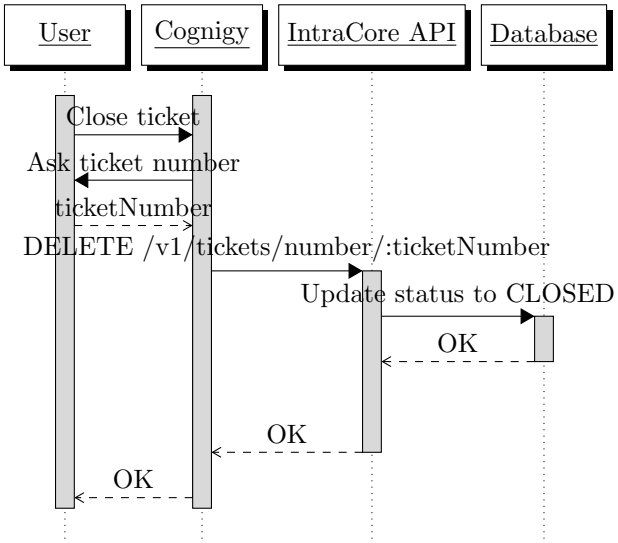

Figure 6: Close ticket: User requests to close ticket; Cognigy asks for ticket number; user provides; Cognigy calls DELETE; API sets status to CLOSED; Cognigy confirms to user.

5.3.7 List Ticket Histories with xApp

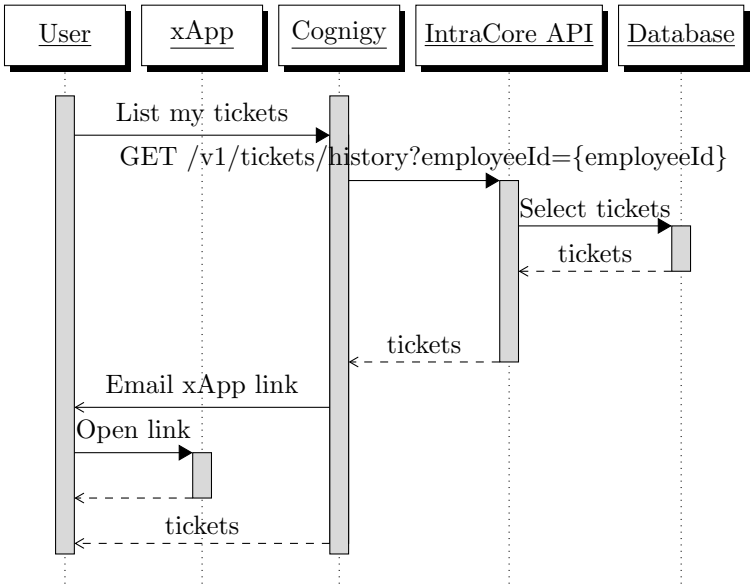

Figure 7: Ticket history: User requests ticket list; Cognigy calls GET /v1/tickets/history with employeeId from verification; API returns tickets; Cognigy emails xApp link; user opens link to view ticket list in xApp.

5.3.8 Escalate Ticket Flow

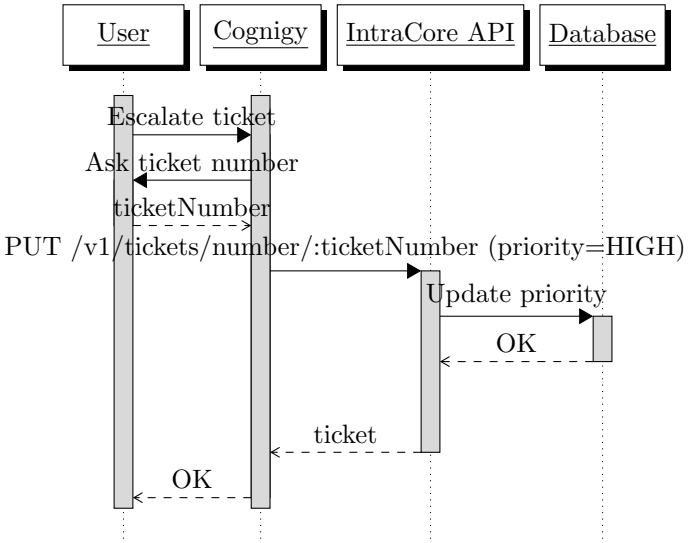

Figure 8: Escalate ticket: User requests escalation; Cognigy asks for ticket number; user provides; Cognigy calls PUT with priority HIGH; API updates ticket; Cognigy confirms to user.

6 Data Model

6.1 Entity Relationship Diagram

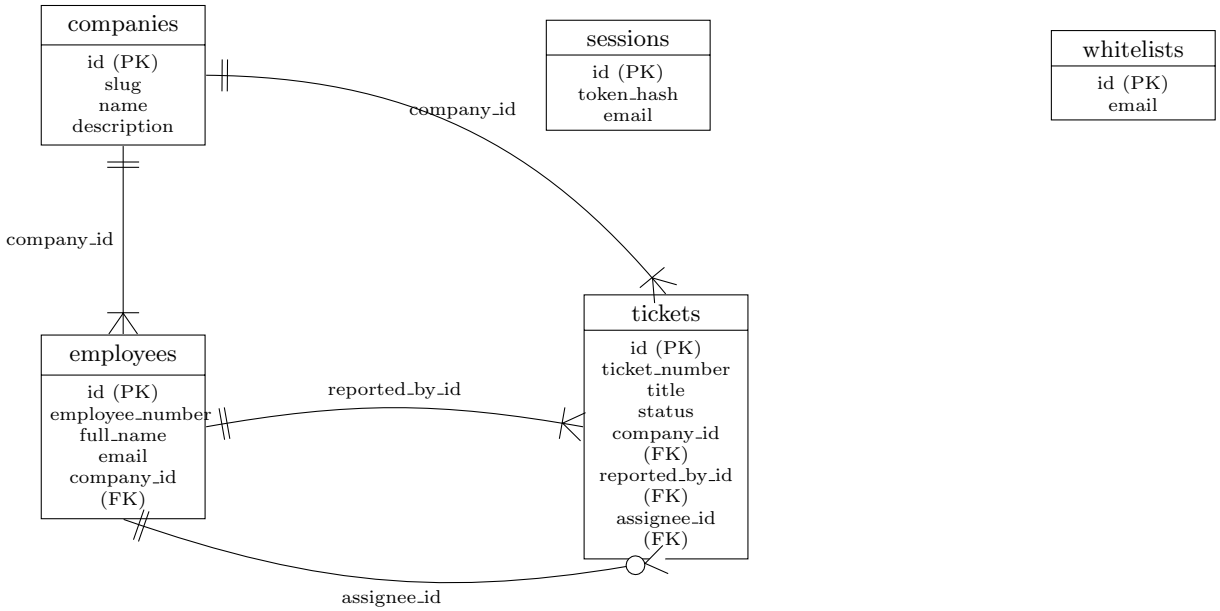

Figure 9: Database entity relationship diagram (crow's foot notation). companies and employees are core entities; tickets references both. sessions and whitelists are standalone (auth).

6.2 companies

```
id          TEXT PRIMARY KEY
slug        TEXT NOT NULL UNIQUE
name        TEXT NOT NULL
description TEXT
created_at  TEXT NOT NULL
updated_at  TEXT NOT NULL
deleted_at  TEXT
```

6.3 employees

```
id                 TEXT PRIMARY KEY
employee_number    TEXT NOT NULL
full_name          TEXT NOT NULL
email              TEXT NOT NULL UNIQUE
phone_number       TEXT NOT NULL
department         TEXT
role               TEXT
preferred_language TEXT NOT NULL DEFAULT 'en-US'
company_id         TEXT NOT NULL REFERENCES companies(id)
created_at         TEXT NOT NULL
updated_at         TEXT NOT NULL
deleted_at         TEXT
UNIQUE(company_id, employee_number)
```

6.4 tickets

```
id              TEXT PRIMARY KEY
ticket_number   TEXT NOT NULL UNIQUE
title           TEXT NOT NULL
description     TEXT
status          TEXT NOT NULL DEFAULT 'NEW'
  -- NEW | PENDING | RESOLVED | CANCELLED | CLOSED
priority        TEXT NOT NULL DEFAULT 'MEDIUM'
  -- LOW | MEDIUM | HIGH
category        TEXT
  -- IT | FACILITIES | MISCELLANEOUS
assignee_id     TEXT REFERENCES employees(id)
reported_by_id  TEXT NOT NULL REFERENCES employees(id)
company_id      TEXT NOT NULL REFERENCES companies(id)
created_at      TEXT NOT NULL
updated_at      TEXT NOT NULL
closed_at       TEXT
```

6.5 sessions

```
id           TEXT PRIMARY KEY
token_hash   TEXT NOT NULL UNIQUE
email        TEXT NOT NULL
expires_at   TEXT NOT NULL
created_at   TEXT NOT NULL
```

6.6 whitelists

```
id          TEXT PRIMARY KEY
email       TEXT NOT NULL UNIQUE
created_at  TEXT NOT NULL
```

7 API Design

7.1 Public Endpoints

- `GET /` — Health/root response
- `GET /health` — Health check
- `POST /v1/auth/token` — Exchange GitHub token for session (body or Bearer)

7.2 Protected Endpoints (Bearer Auth)

- `POST /v1/auth/verify` — Identity verification (phoneNumber, email, employeeNumber)
- `GET /v1/tickets` — List tickets
- `POST /v1/tickets` — Create ticket
- `GET /v1/tickets/:id` — Get ticket by ID
- `PUT /v1/tickets/:id` — Update ticket
- `DELETE /v1/tickets/:id` — Close ticket
- `GET /v1/tickets/number/:ticketNumber` — Get ticket by ticket number
- `PUT /v1/tickets/number/:ticketNumber` — Update ticket by number
- `DELETE /v1/tickets/number/:ticketNumber` — Close ticket by number
- `GET /v1/tickets/history?employeeId=`{`employeeId`} — Ticket history for employee

- `GET|POST /v1/companies`, `GET|PUT|DELETE /v1/companies/:id`
- `GET|POST /v1/employees`, `GET|PUT|DELETE /v1/employees/:id`
- `GET|POST /v1/whitelists`, `GET|PUT|DELETE /v1/whitelists/:id`
- `GET /v1/sessions`, `GET|DELETE /v1/sessions/:id`

7.3 Use Case to API Mapping

Use Case	API
ID&V	POST /v1/auth/verify
Open ticket	POST /v1/tickets
Check status	GET /v1/tickets/number/:ticketNumber or GET /v1/tickets/:id
Update ticket	PUT /v1/tickets/:id or PUT /v1/tickets/number/:ticketNumber
Close ticket	DELETE /v1/tickets/:id or DELETE /v1/tickets/number/:ticketNumber
Search history	GET /v1/tickets/history?employeeId={employeeId}

7.4 OpenAPI

Interactive API documentation: https://intracore.eden.spicyz.io/docs

8 Identity and Verification

`POST /v1/auth/verify` requires a valid Bearer token and accepts:

```
{
  "phoneNumber": "+1234567890",
  "email": "user@example.com",
  "employeeNumber": "007"
}
```

The API matches all three fields against the `employees` table (excluding soft-deleted rows). On success it returns:

```
{
  "id": "uuid",
  "fullName": "Jane Doe",
  "email": "user@example.com",
  "preferredLanguage": "en-US"
}
```

On failure: 401 with {`"error":"Unauthorized","message":"Identity verification failed"`}.

9 Authentication

1. User authenticates with GitHub OAuth (outside this API)
2. Client sends GitHub token to `POST /v1/auth/token` (in body as `githubToken` or in `Authorization: Bearer <token>`)
3. API validates GitHub token and checks email against `whitelists`
4. If whitelisted, API creates a session and returns `accessToken`, `expiresAt`, `tokenType: "Bearer"`
5. All protected routes require `Authorization: Bearer <accessToken>`

10 Cognigy xApps

Three xApps are provided, built with the Cognigy app-page SDK and Material Web:

- **verification.html** — Collects phone number, email, employee number; submits via `SDK.submit({phoneNumber, email, employeeNumber})`
- **ticket-list.html** — Displays ticket history with status/priority badges; supports English, Thai, Japanese, German
- **ticket-detail.html** — Shows full ticket details (status, priority, category, description, company, reporter, assignee, dates)

All xApps use `/sdk/app-page-sdk.js`, Material Web components (`md-outlined-text-field`, `md-filled-button`, `md-outlined-select`), and multilingual labels.

11 Deployment

11.1 Docker

Multi-stage build:

```
# Stage 1: Build
FROM oven/bun:1 AS builder
WORKDIR /app
COPY package.json bun.lock ./
RUN bun install --frozen-lockfile
COPY . .
RUN bun build --compile --minify server.ts --outfile intracore

# Stage 2: Runtime
FROM gcr.io/distroless/cc-debian12:nonroot
WORKDIR /app
COPY --from=builder --chown=65532:65532 /app/intracore .
EXPOSE 3000
ENV PORT=3000
CMD ["/app/intracore"]
```

11.2 Environment

- `DATABASE_URL` — Postgres connection string
- `PORT` — Server port (default 3000)
- `SESSION_TTL_SECONDS` — Session lifetime in seconds (default 900)

12 CI/CD

GitHub Actions workflow `.github/workflows/docker-publish.yml`:

- Triggers on push to `main`
- Builds and pushes Docker image

13 Security

- **Token hashing:** Session tokens are hashed (SHA-256) before storage; only the hash is stored
- **Whitelist:** Only whitelisted emails can obtain session tokens
- **HTTPS:** TLS termination at Cloudflare

14 Deliverables Summary

- Source code: https://github.com/boss-spicyz100x/intracore
- Cognigy platform flow design (voice agent, xApps)
- IntraCore API integration for ID&V and ticketing
- xApp (verification, ticket-list, ticket-detail)
- Multilingual support (English, Thai, Japanese, German)
- Presentations and documentation for Capstone Sprint evaluation